Phu Nguyen

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/healthcare-dataset-stroke-data.csv")
print(df.shape)
df.head()
```

```
(5110, 12)
```

Out[1]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_t
0	9046	Male	67.0	0	1	Yes	Private	Ur
1	51676	Female	61.0	0	0	Yes	Self-employed	R
2	31112	Male	80.0	0	1	Yes	Private	R
3	60182	Female	49.0	0	0	Yes	Private	Ur
4	1665	Female	79.0	1	0	Yes	Self-employed	R

In [2]:
```
train_df, test_df = train_test_split(
    df, test_size=0.30, random_state=42, stratify=df["stroke"]
)
print("Train:", train_df.shape, "Test:", test_df.shape)

train_df.to_csv("data/train_raw.csv", index=False)
test_df.to_csv("data/test_raw.csv", index=False)
```

```
Train: (3577, 12) Test: (1533, 12)
```

In [3]:
```
print(train_df.info())
print()
print("Missing values per column:")
print(train_df.isnull().sum())
```

```
<class 'pandas.DataFrame'>
Index: 3577 entries, 2226 to 5052
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 3577 non-null   int64  
 1   gender             3577 non-null   str    
 2   age                3577 non-null   float64
 3   hypertension       3577 non-null   int64  
 4   heart_disease      3577 non-null   int64  
 5   ever_married       3577 non-null   str    
 6   work_type          3577 non-null   str    
 7   Residence_type     3577 non-null   str    
 8   avg_glucose_level  3577 non-null   float64
 9   bmi                3429 non-null   float64
 10  smoking_status     3577 non-null   str    
 11  stroke             3577 non-null   int64  
dtypes: float64(3), int64(4), str(5)
memory usage: 363.3 KB
None

Missing values per column:
id                     0
gender                 0
age                    0
hypertension           0
heart_disease          0
ever_married           0
work_type              0
Residence_type         0
avg_glucose_level      0
bmi                  148
smoking_status         0
stroke                 0
dtype: int64
```

In [4]:
```
print("Duplicate rows:", train_df.duplicated().sum())
print()
print(train_df.describe())
print()
for col in ["gender", "ever_married", "work_type", "Residence_type", "smoking_statu
    print(train_df[col].value_counts())
    print()
```

```
Duplicate rows: 0

                 id          age  hypertension  heart_disease  \
count   3577.000000  3577.000000   3577.000000    3577.000000
mean   36557.110428    43.250433      0.096170       0.053956
std    21295.347024    22.493077      0.294865       0.225962
min       77.000000     0.080000      0.000000       0.000000
25%    17441.000000    26.000000      0.000000       0.000000
50%    36969.000000    45.000000      0.000000       0.000000
75%    54815.000000    61.000000      0.000000       0.000000
max    72918.000000    82.000000      1.000000       1.000000

       avg_glucose_level          bmi       stroke
count        3577.000000  3429.000000  3577.000000
mean          105.957414    28.867279     0.048644
std            44.945203     7.909601     0.215153
min            55.120000    10.300000     0.000000
25%            77.460000    23.500000     0.000000
50%            91.710000    28.000000     0.000000
75%           113.630000    33.100000     0.000000
max           271.740000    92.000000     1.000000

gender
Female    2093
Male      1483
Other        1
Name: count, dtype: int64

ever_married
Yes    2361
No     1216
Name: count, dtype: int64

work_type
Private          2044
Self-employed     589
children          481
Govt_job          451
Never_worked       12
Name: count, dtype: int64

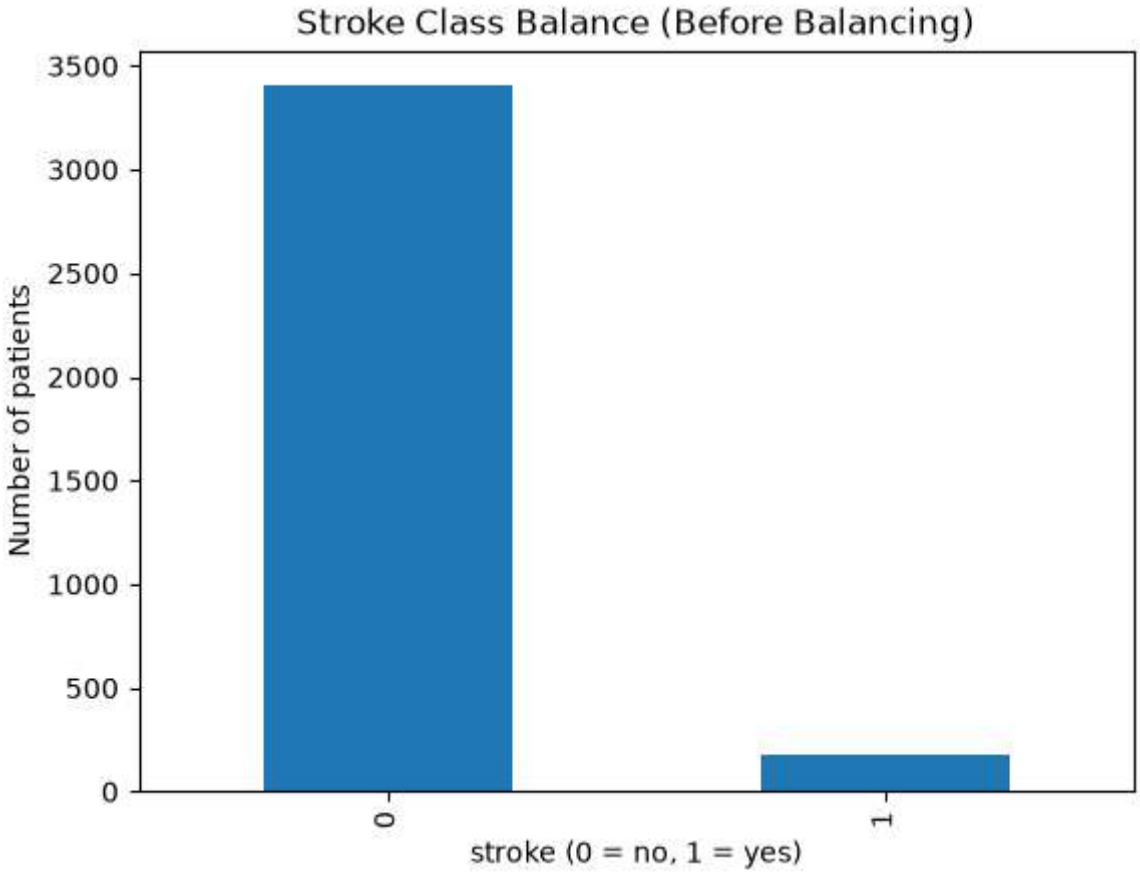
Residence_type
Urban    1803
Rural    1774
Name: count, dtype: int64

smoking_status
never smoked       1327
Unknown            1084
formerly smoked     631
smokes              535
Name: count, dtype: int64
```

In [5]:
```
train_df["stroke"].value_counts().plot(kind="bar")
plt.title("Stroke Class Balance (Before Balancing)")
```

```
plt.xlabel("stroke (0 = no, 1 = yes)")
plt.ylabel("Number of patients")
plt.show()

print(train_df["stroke"].value_counts(normalize=True) * 100)
```




```
stroke
0    95.135588
1     4.864412
Name: proportion, dtype: float64
```

In [6]:
```
# Work on a copy so train_df stays as the "before" version
clean_df = train_df.copy()

print("Missing bmi BEFORE:", clean_df["bmi"].isnull().sum())

fig, ax = plt.subplots(1, 2, figsize=(10, 4))

clean_df["bmi"].hist(ax=ax[0], bins=30)
ax[0].set_title("BMI before filling")

median_bmi = clean_df["bmi"].median()
clean_df["bmi"] = clean_df["bmi"].fillna(median_bmi)

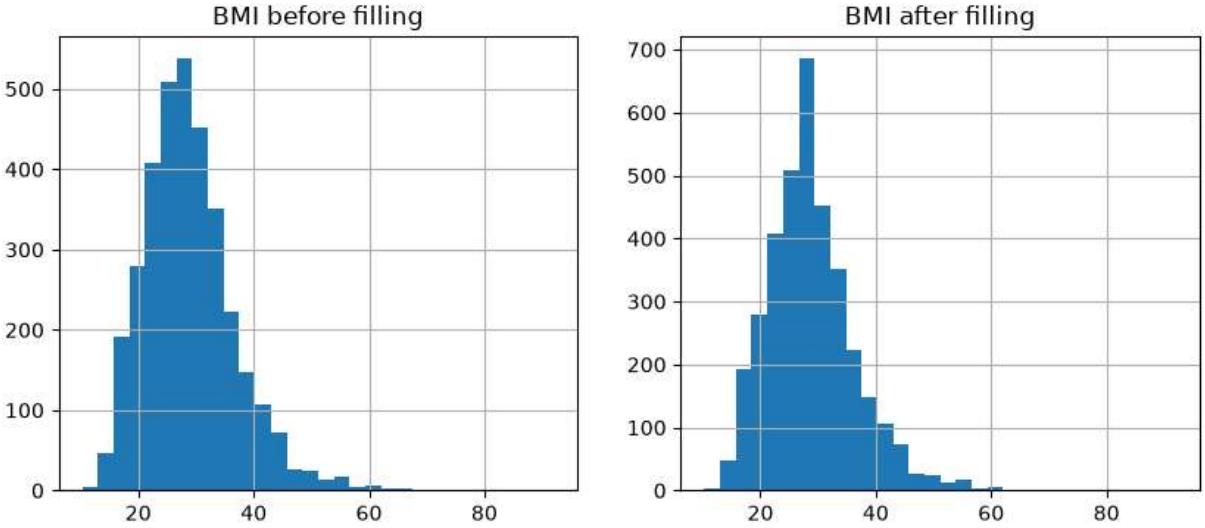
clean_df["bmi"].hist(ax=ax[1], bins=30)
ax[1].set_title("BMI after filling")

plt.show()

print("Median used:", median_bmi)
```

```
print("Missing bmi AFTER:", clean_df["bmi"].isnull().sum())
clean_df.head()
```

```
Missing bmi BEFORE: 148
```




```
Median used: 28.0
Missing bmi AFTER: 0
```

Out[6]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residen
2226	35602	Female	52.0	0	0	Yes	Govt_job	
3927	56137	Female	62.0	0	0	Yes	Private	
3358	14063	Male	81.0	0	1	No	Self-employed	
4152	12857	Male	55.0	0	0	Yes	Self-employed	
4866	5875	Female	37.0	0	0	Yes	Private	

In [7]:

```
import numpy as np

print("BEFORE:")
print(clean_df["smoking_status"].value_counts())

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
clean_df["smoking_status"].value_counts().plot(kind="bar", ax=ax[0])
ax[0].set_title("smoking_status before")

# Turn "Unknown" into a real missing value, then fill with the most common value
clean_df["smoking_status"] = clean_df["smoking_status"].replace("Unknown", np.nan)
print("\nMissing after replacing Unknown:", clean_df["smoking_status"].isnull().sum

mode_value = clean_df["smoking_status"].mode()[0]
clean_df["smoking_status"] = clean_df["smoking_status"].fillna(mode_value)

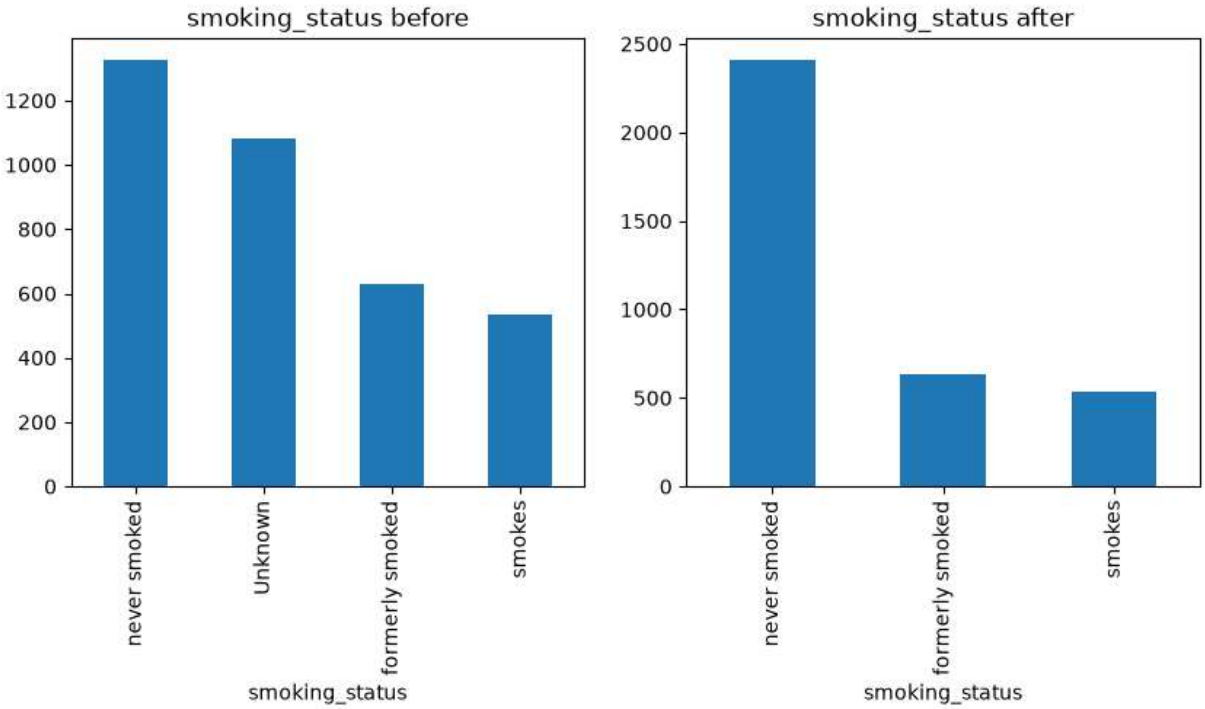
clean_df["smoking_status"].value_counts().plot(kind="bar", ax=ax[1])
ax[1].set_title("smoking_status after")
```

```
plt.show()

print("\nMode used:", mode_value)
print("\nAFTER:")
print(clean_df["smoking_status"].value_counts())
```

```
BEFORE:
smoking_status
never smoked       1327
Unknown            1084
formerly smoked     631
smokes              535
Name: count, dtype: int64

Missing after replacing Unknown: 1084
```




```
Mode used: never smoked

AFTER:
smoking_status
never smoked       2411
formerly smoked     631
smokes              535
Name: count, dtype: int64
```

In [8]:
```
print("Shape BEFORE:", clean_df.shape)
print(clean_df["gender"].value_counts())

# Drop the id column
clean_df = clean_df.drop(columns=["id"])

# Remove the row where gender is "Other"
clean_df = clean_df[clean_df["gender"] != "Other"]

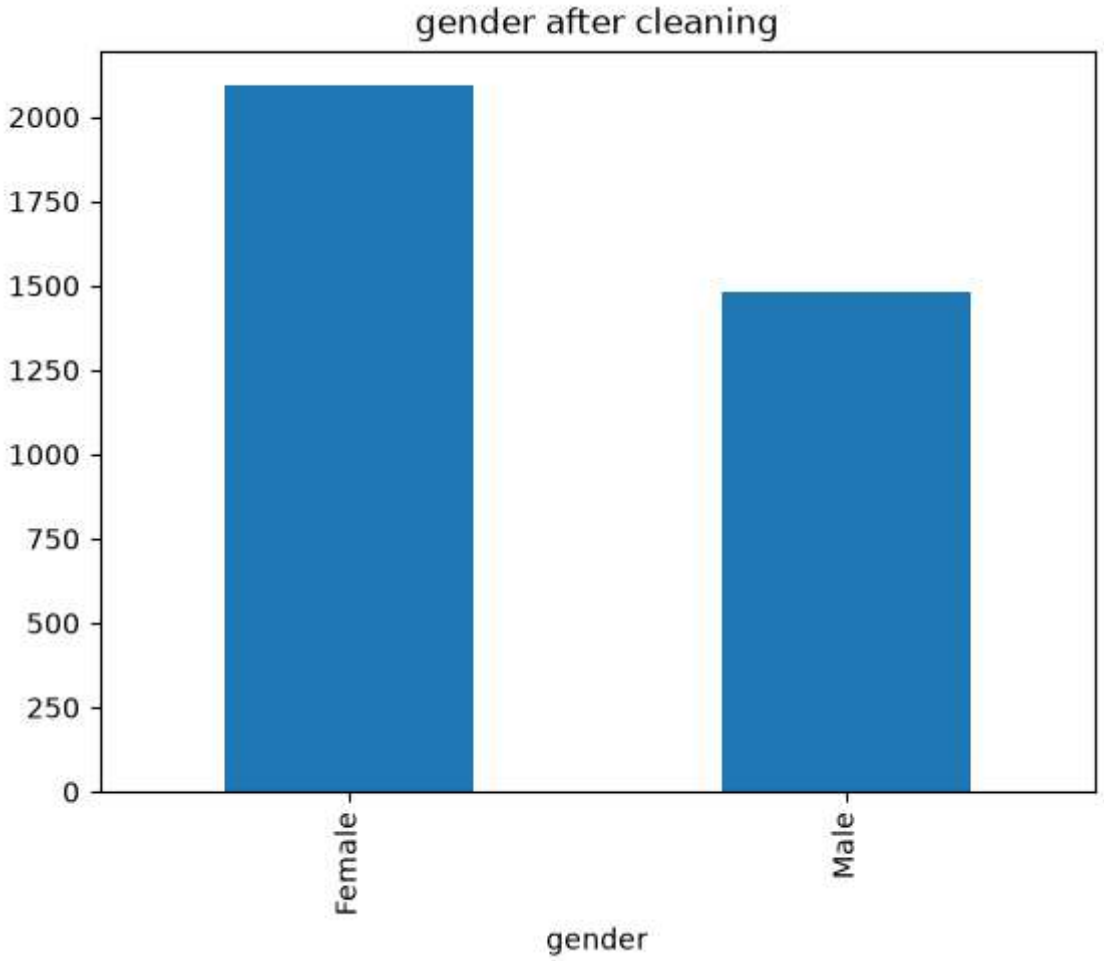
print("\nShape AFTER:", clean_df.shape)
print(clean_df["gender"].value_counts())
```

```
clean_df["gender"].value_counts().plot(kind="bar")
plt.title("gender after cleaning")
plt.show()

clean_df.head()
```

```
Shape BEFORE: (3577, 12)
gender
Female    2093
Male      1483
Other        1
Name: count, dtype: int64

Shape AFTER: (3576, 11)
gender
Female    2093
Male      1483
Name: count, dtype: int64
```



Out[8]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type
2226	Female	52.0	0	0	Yes	Govt_job	Rural
3927	Female	62.0	0	0	Yes	Private	Urban
3358	Male	81.0	0	1	No	Self-employed	Rural
4152	Male	55.0	0	0	Yes	Self-employed	Rural
4866	Female	37.0	0	0	Yes	Private	Urban

In [9]:

```
cols = ["bmi", "avg_glucose_level"]

fig, ax = plt.subplots(2, 2, figsize=(10, 8))

for i, col in enumerate(cols):
    ax[0][i].boxplot(clean_df[col], vert=False)
    ax[0][i].set_title(f"{col} BEFORE")

    q1 = clean_df[col].quantile(0.25)
    q3 = clean_df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    n_out = ((clean_df[col] < lower) | (clean_df[col] > upper)).sum()
    print(f"{col}: lower={lower:.2f}, upper={upper:.2f}, outliers={n_out}")

    clean_df[col] = clean_df[col].clip(lower, upper)

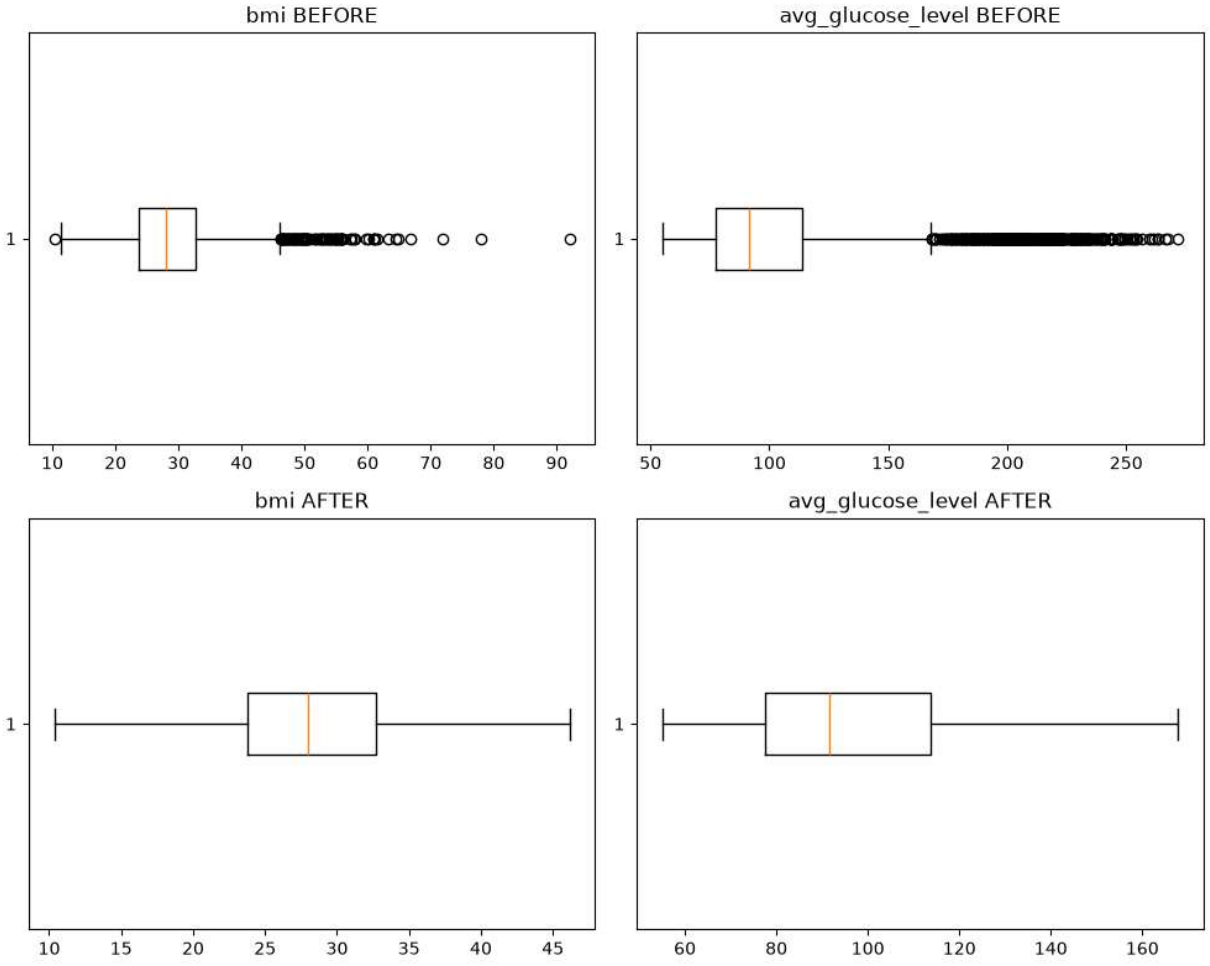
    ax[1][i].boxplot(clean_df[col], vert=False)
    ax[1][i].set_title(f"{col} AFTER")

plt.tight_layout()
plt.show()

print(clean_df[cols].describe())
```

```
C:\Users\PhuNguyen\AppData\Local\Temp\ipykernel_8160\2567635377.py:6: MatplotlibDepr
ecationWarning: vert: bool was deprecated in Matplotlib 3.11 and will be removed in
3.13. Use orientation: {'vertical', 'horizontal'} instead.
  ax[0][i].boxplot(clean_df[col], vert=False)
C:\Users\PhuNguyen\AppData\Local\Temp\ipykernel_8160\2567635377.py:20: MatplotlibDep
recationWarning: vert: bool was deprecated in Matplotlib 3.11 and will be removed in
3.13. Use orientation: {'vertical', 'horizontal'} instead.
  ax[1][i].boxplot(clean_df[col], vert=False)
C:\Users\PhuNguyen\AppData\Local\Temp\ipykernel_8160\2567635377.py:6: MatplotlibDepr
ecationWarning: vert: bool was deprecated in Matplotlib 3.11 and will be removed in
3.13. Use orientation: {'vertical', 'horizontal'} instead.
  ax[0][i].boxplot(clean_df[col], vert=False)
C:\Users\PhuNguyen\AppData\Local\Temp\ipykernel_8160\2567635377.py:20: MatplotlibDep
recationWarning: vert: bool was deprecated in Matplotlib 3.11 and will be removed in
3.13. Use orientation: {'vertical', 'horizontal'} instead.
  ax[1][i].boxplot(clean_df[col], vert=False)
```

```
bmi: lower=10.35, upper=46.15, outliers=95
avg_glucose_level: lower=23.27, upper=167.78, outliers=436
```



	bmi	avg_glucose_level
count	3576.000000	3576.000000
mean	28.648839	100.702283
std	7.140682	32.644429
min	10.350000	55.120000
25%	23.775000	77.457500
50%	28.000000	91.695000
75%	32.725000	113.585000
max	46.150000	167.776250

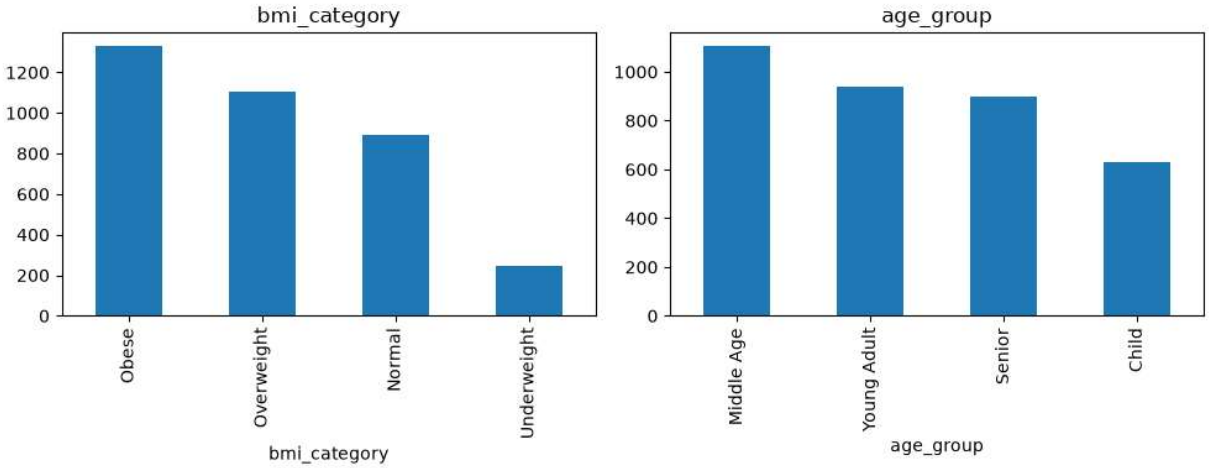
In [10]:
```
# BMI category (standard health ranges)
clean_df["bmi_category"] = pd.cut(
    clean_df["bmi"],
    bins=[0, 18.5, 25, 30, 100],
    labels=["Underweight", "Normal", "Overweight", "Obese"]
)

# Age group
clean_df["age_group"] = pd.cut(
    clean_df["age"],
    bins=[0, 18, 40, 60, 120],
    labels=["Child", "Young Adult", "Middle Age", "Senior"]
)

print(clean_df[["bmi", "bmi_category", "age", "age_group"]].head(10))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
clean_df["bmi_category"].value_counts().plot(kind="bar", ax=ax[0])
ax[0].set_title("bmi_category")
clean_df["age_group"].value_counts().plot(kind="bar", ax=ax[1])
ax[1].set_title("age_group")
plt.tight_layout()
plt.show()
```

```
       bmi bmi_category   age    age_group
2226  22.0       Normal  52.0   Middle Age
3927  36.3        Obese  62.0       Senior
3358  29.4   Overweight  81.0       Senior
4152  28.0   Overweight  55.0   Middle Age
4866  36.1        Obese  37.0  Young Adult
2779  19.8       Normal   6.0        Child
1299  35.3        Obese  51.0   Middle Age
853   36.6        Obese  40.0  Young Adult
5020  17.6  Underweight   5.0        Child
3363  43.4        Obese  61.0       Senior
```



In [11]:
```
print("BEFORE encoding:")
print(clean_df[["gender", "ever_married", "Residence_type", "work_type", "smoking_s

# 1) Simple encoding for 2-option columns
clean_df["gender"] = clean_df["gender"].map({"Male": 0, "Female": 1})
```

```
clean_df["ever_married"] = clean_df["ever_married"].map({"No": 0, "Yes": 1})
clean_df["Residence_type"] = clean_df["Residence_type"].map({"Rural": 0, "Urban": 1

# 2) One-hot encoding for columns with 3+ options
clean_df = pd.get_dummies(
    clean_df,
    columns=["work_type", "smoking_status", "bmi_category", "age_group"],
    dtype=int
)

print("\nAFTER encoding:")
print(clean_df.shape)
print(clean_df.columns.tolist())
clean_df.head()
```

```
BEFORE encoding:
      gender ever_married Residence_type      work_type   smoking_status
2226  Female          Yes          Rural       Govt_job  formerly smoked
3927  Female          Yes          Urban        Private     never smoked
3358    Male           No          Rural  Self-employed     never smoked
4152    Male          Yes          Rural  Self-employed           smokes
4866  Female          Yes          Urban        Private           smokes

AFTER encoding:
(3576, 25)
['gender', 'age', 'hypertension', 'heart_disease', 'ever_married', 'Residence_type',
'avg_glucose_level', 'bmi', 'stroke', 'work_type_Govt_job', 'work_type_Never_worke
d', 'work_type_Private', 'work_type_Self-employed', 'work_type_children', 'smoking_s
tatus_formerly smoked', 'smoking_status_never smoked', 'smoking_status_smokes', 'bmi
_category_Underweight', 'bmi_category_Normal', 'bmi_category_Overweight', 'bmi_categ
ory_Obese', 'age_group_Child', 'age_group_Young Adult', 'age_group_Middle Age', 'age
_group_Senior']
```

Out[11]:

	gender	age	hypertension	heart_disease	ever_married	Residence_type	avg_glucos
2226	1	52.0	0	0	1	0	
3927	1	62.0	0	0	1	1	
3358	0	81.0	0	1	0	0	
4152	0	55.0	0	0	1	0	
4866	1	37.0	0	0	1	1	

5 rows × 25 columns

In [12]:

```
import numpy as np

print("Skew BEFORE:", round(clean_df["avg_glucose_level"].skew(), 3))

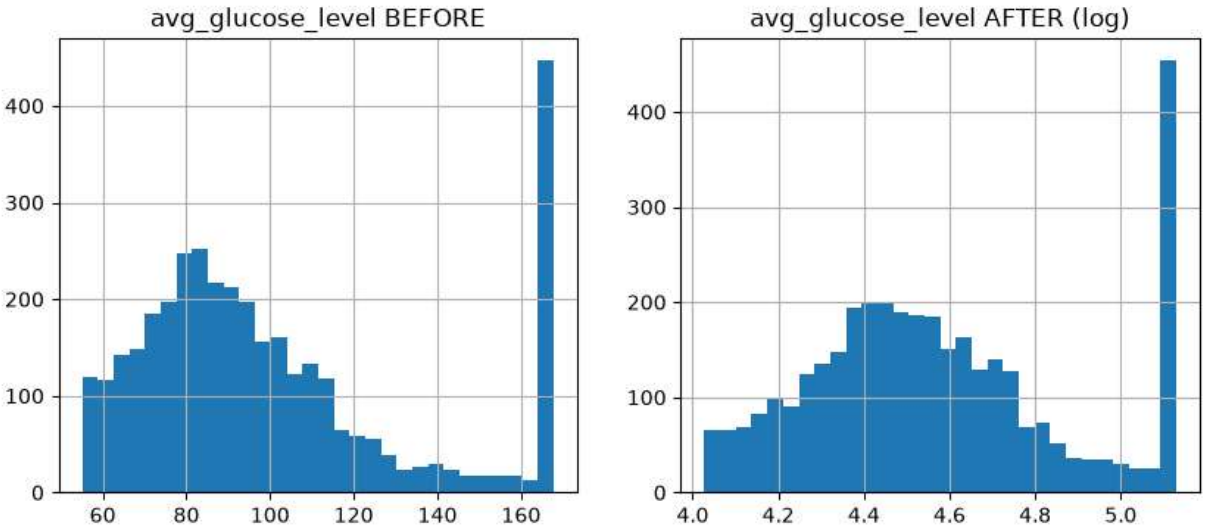
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
clean_df["avg_glucose_level"].hist(ax=ax[0], bins=30)
ax[0].set_title("avg_glucose_level BEFORE")
```

```
clean_df["avg_glucose_level"] = np.log1p(clean_df["avg_glucose_level"])

clean_df["avg_glucose_level"].hist(ax=ax[1], bins=30)
ax[1].set_title("avg_glucose_level AFTER (log)")
plt.show()

print("Skew AFTER:", round(clean_df["avg_glucose_level"].skew(), 3))
clean_df[["avg_glucose_level"]].head()
```

```
Skew BEFORE: 0.923
```




```
Skew AFTER: 0.439
```

Out[12]:

	avg_glucose_level
2226	4.689879
3927	4.492225
3358	4.569439
4152	4.311738
4866	4.650717

In [13]:

```
from sklearn.preprocessing import MinMaxScaler

num_cols = ["age", "avg_glucose_level", "bmi"]

print("BEFORE scaling:")
print(clean_df[num_cols].describe().loc[["min", "max"]])

fig, ax = plt.subplots(2, 3, figsize=(12, 6))
for i, col in enumerate(num_cols):
    clean_df[col].hist(ax=ax[0][i], bins=30)
    ax[0][i].set_title(f"{col} BEFORE")

scaler = MinMaxScaler()
clean_df[num_cols] = scaler.fit_transform(clean_df[num_cols])

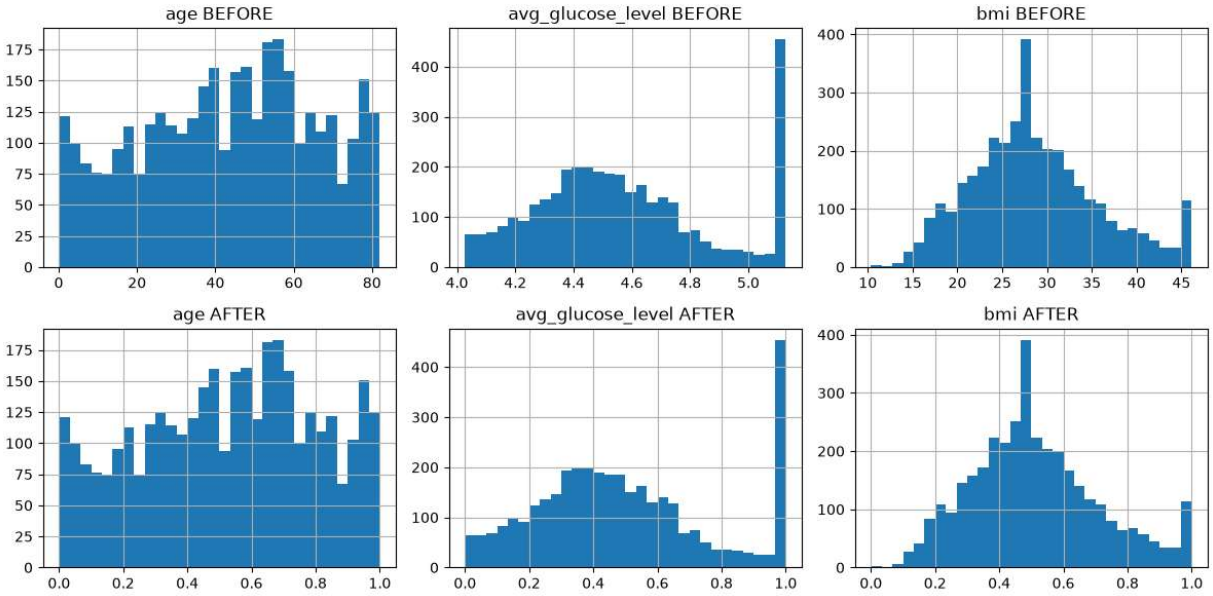
for i, col in enumerate(num_cols):
    clean_df[col].hist(ax=ax[1][i], bins=30)
```

```
    ax[1][i].set_title(f"{col} AFTER")

plt.tight_layout()
plt.show()

print("\nAFTER scaling:")
print(clean_df[num_cols].describe().loc[["min", "max"]])
clean_df[num_cols].head()
```

```
BEFORE scaling:
       age  avg_glucose_level    bmi
min   0.08           4.027492  10.35
max  82.00           5.128574  46.15
```




```
AFTER scaling:
     age  avg_glucose_level  bmi
min  0.0                0.0  0.0
max  1.0                1.0  1.0
```

Out[13]:

	age	avg_glucose_level	bmi
2226	0.633789	0.601578	0.325419
3927	0.755859	0.422070	0.724860
3358	0.987793	0.492195	0.532123
4152	0.670410	0.258152	0.493017
4866	0.450684	0.566011	0.719274

In [14]:

```
from sklearn.utils import resample

print("BEFORE balancing:")
print(clean_df["stroke"].value_counts())

majority = clean_df[clean_df["stroke"] == 0]
minority = clean_df[clean_df["stroke"] == 1]

minority_upsampled = resample(
```

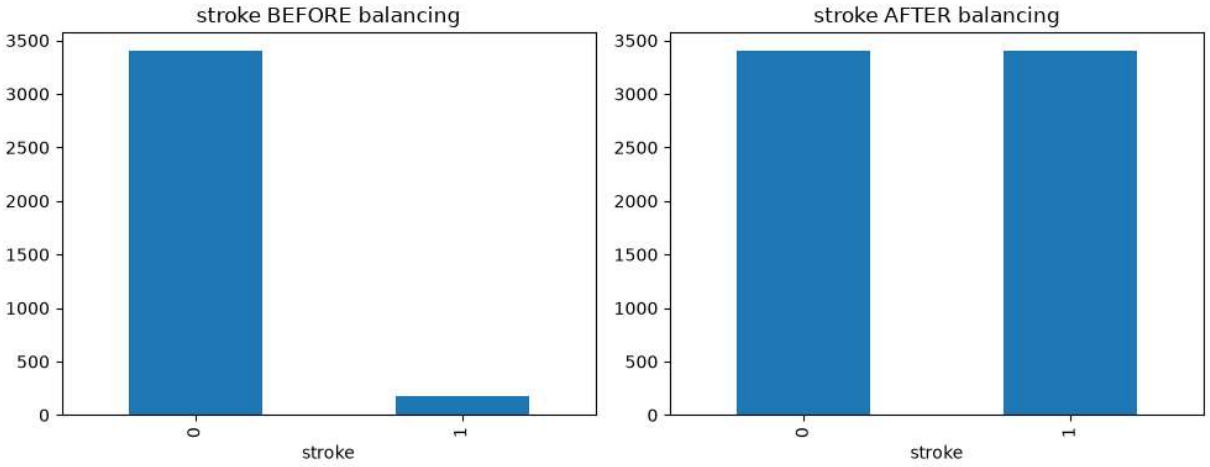
```
    minority,
    replace=True,
    n_samples=len(majority),
    random_state=42
)

balanced_df = pd.concat([majority, minority_upsampled])
balanced_df = balanced_df.sample(frac=1, random_state=42).reset_index(drop=True)

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
clean_df["stroke"].value_counts().plot(kind="bar", ax=ax[0])
ax[0].set_title("stroke BEFORE balancing")
balanced_df["stroke"].value_counts().plot(kind="bar", ax=ax[1])
ax[1].set_title("stroke AFTER balancing")
plt.tight_layout()
plt.show()

print("\nAFTER balancing:")
print(balanced_df["stroke"].value_counts())
print(balanced_df.shape)
```

```
BEFORE balancing:
stroke
0    3402
1     174
Name: count, dtype: int64
```




```
AFTER balancing:
stroke
0    3402
1    3402
Name: count, dtype: int64
(6804, 25)
```

In [15]:
```
balanced_df.to_csv("data/final_clean_train.csv", index=False)

print("Original data:      ", df.shape)
print("Training split raw: ", train_df.shape)
print("Final clean data:   ", balanced_df.shape)
print("Missing values left:", balanced_df.isnull().sum().sum())
balanced_df.head()
```

```
Original data:       (5110, 12)
Training split raw:  (3577, 12)
Final clean data:    (6804, 25)
Missing values left: 0
```

Out[15]:

	gender	age	hypertension	heart_disease	ever_married	Residence_type	avg_gluco
0	1	0.707031	0	0	1	1	0
1	1	0.963379	0	1	1	1	1
2	1	0.157715	0	0	0	1	0
3	1	0.645996	0	0	1	0	0
4	1	0.682617	0	0	1	1	0

5 rows × 25 columns

In [16]:

```
readable_df = train_df.copy()

# Fill missing bmi with the median
readable_df["bmi"] = readable_df["bmi"].fillna(readable_df["bmi"].median())

# Turn "Unknown" smoking into missing, then fill with the most common value
readable_df["smoking_status"] = readable_df["smoking_status"].replace("Unknown", np
readable_df["smoking_status"] = readable_df["smoking_status"].fillna(readable_df["s

# Drop id and the single "Other" gender row
readable_df = readable_df.drop(columns=["id"])
readable_df = readable_df[readable_df["gender"] != "Other"]

# Cap outliers with the IQR rule
for col in ["bmi", "avg_glucose_level"]:
    q1 = readable_df[col].quantile(0.25)
    q3 = readable_df[col].quantile(0.75)
    iqr = q3 - q1
    readable_df[col] = readable_df[col].clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)

# Feature engineering
readable_df["bmi_category"] = pd.cut(
    readable_df["bmi"], bins=[0, 18.5, 25, 30, 100],
    labels=["Underweight", "Normal", "Overweight", "Obese"]
)
readable_df["age_group"] = pd.cut(
    readable_df["age"], bins=[0, 18, 40, 60, 120],
    labels=["Child", "Young Adult", "Middle Age", "Senior"]
)

# Balance the classes
majority = readable_df[readable_df["stroke"] == 0]
minority = readable_df[readable_df["stroke"] == 1]
minority_up = resample(minority, replace=True, n_samples=len(majority), random_stat
readable_df = pd.concat([majority, minority_up]).sample(frac=1, random_state=42).re
```

```
readable_df.to_csv("data/final_clean_train_readable.csv", index=False)

print(readable_df.shape)
print(readable_df["stroke"].value_counts())
readable_df.head()
```

```
(6804, 13)
stroke
0    3402
1    3402
Name: count, dtype: int64
```

Out[16]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	av
0	Female	58.0	0	0	Yes	Private	Urban	
1	Female	79.0	0	1	Yes	Private	Urban	
2	Female	13.0	0	0	No	Private	Urban	
3	Female	53.0	0	0	Yes	Private	Rural	
4	Female	56.0	0	0	Yes	Private	Urban	

In [17]:

```
readable_df["age"] = readable_df["age"].round().astype(int)

# Optional: tidy the other decimals too
readable_df["bmi"] = readable_df["bmi"].round(1)
readable_df["avg_glucose_level"] = readable_df["avg_glucose_level"].round(1)

readable_df.to_csv("data/final_clean_train_readable.csv", index=False)

print(readable_df["age"].describe())
readable_df.head()
```

```
count    6804.000000
mean       54.996914
std        22.060555
min         0.000000
25%        42.000000
50%        59.000000
75%        74.000000
max        82.000000
Name: age, dtype: float64
```

Out[17]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	av
0	Female	58	0	0	Yes	Private	Urban	
1	Female	79	0	1	Yes	Private	Urban	
2	Female	13	0	0	No	Private	Urban	
3	Female	53	0	0	Yes	Private	Rural	
4	Female	56	0	0	Yes	Private	Urban	

In [18]:
```
print(df["bmi"].isnull().sum())
```

```
201
```

In [ ]: